МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт Информационных Технологий
Кафедра Вычислительной Техники (ВТ)

ОТЧЁТ ПО БЛОКУ ЗАДАНИЙ № 1

по дисциплине

«СХЕМОТЕХНИКА УСТРОЙСТВ КОМПЬЮТЕРНЫХ СИСТЕМ»

Выполнил студент группы *Дьяков А. Г.*

ИВБО-20-24

Принял преподаватель кафедры ВТ Дуксина И. И.

Блок заданий выполнен « __ » _____ 2026 г.

«Зачтено» « __ » _____ 2026 г.

Москва 2026

СОДЕРЖАНИЕ

1 ПРОЕКТИРОВАНИЕ 4

1.1 Алгоритм 4

1.2 Блок-схема 5

1.3 Описание блок-схемы 6

1.4 Компонентный состав (аппаратные ресурсы) 6

1.5 АСК — архитектура системы команд 6

1.6 Микроархитектура (команды по тактам) 7

1.7 Основные размерности 7

1.8 Ассемблер 8

1.9 Python-эмулятор 9

2 ТЕСТИРОВАНИЕ 11

2.1 Результаты тестирования 12

ЗАКЛЮЧЕНИЕ 13

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 14

ВВЕДЕНИЕ

Целью данной работы является разработка компонентного состава, архитектуры системы команд (АСК) и эмулятора процессорного ядра для решения индивидуального варианта задачи: даны натуральные числа m и n, требуется получить все натуральные числа, меньшие n, квадрат суммы цифр которых равен m.

Работа выполняется в два этапа. На аналитическом этапе разрабатывается алгоритм решения задачи, строится блок-схема, определяется набор аппаратных ресурсов (регистровый файл, память команд, память данных, АЛУ и вспомогательные регистры) и система команд, необходимая и достаточная для реализации алгоритма на уровне регистровых пересылок. Для каждой используемой команды определяется её функциональное назначение, формат аргументов, порядок выполнения по тактам (Fetch — Decode — Execute) и разрядность.

На втором этапе на основе принятой аналитики реализуется программный эмулятор процессорного ядра на языке высокого уровня (Python), а также ассемблерная программа, реализующая алгоритм в терминах разработанной системы команд. Особенность данного варианта в том, что система команд не содержит операций умножения и деления, поэтому вычисление суммы цифр числа (через выделение цифр операцией деления по модулю) и возведение суммы в квадрат должны быть сведены к последовательности допустимых операций — вычитания, сложения, инкремента и условных переходов.

1 ПРОЕКТИРОВАНИЕ

1.1 Алгоритм

Вариант: даны натуральные числа m и n. Получить все натуральные числа, меньшие n, квадрат суммы цифр которых равен m.

Листинг 1.1 — Код алгоритма на Python

```
def find_numbers(n, m):
    result = []
    k = 1
    while k < n:
        x = k
        s = 0
        while x > 0:
            d = x % 10
            s = s + d
            x = x // 10
        sq = s * s
        if sq == m:
            result.append(k)
        k = k + 1
    return result


if __name__ == '__main__':
    n = int(input("n = "))
    m = int(input("m = "))
    print(find_numbers(n, m))
```

1.2 Блок-схема

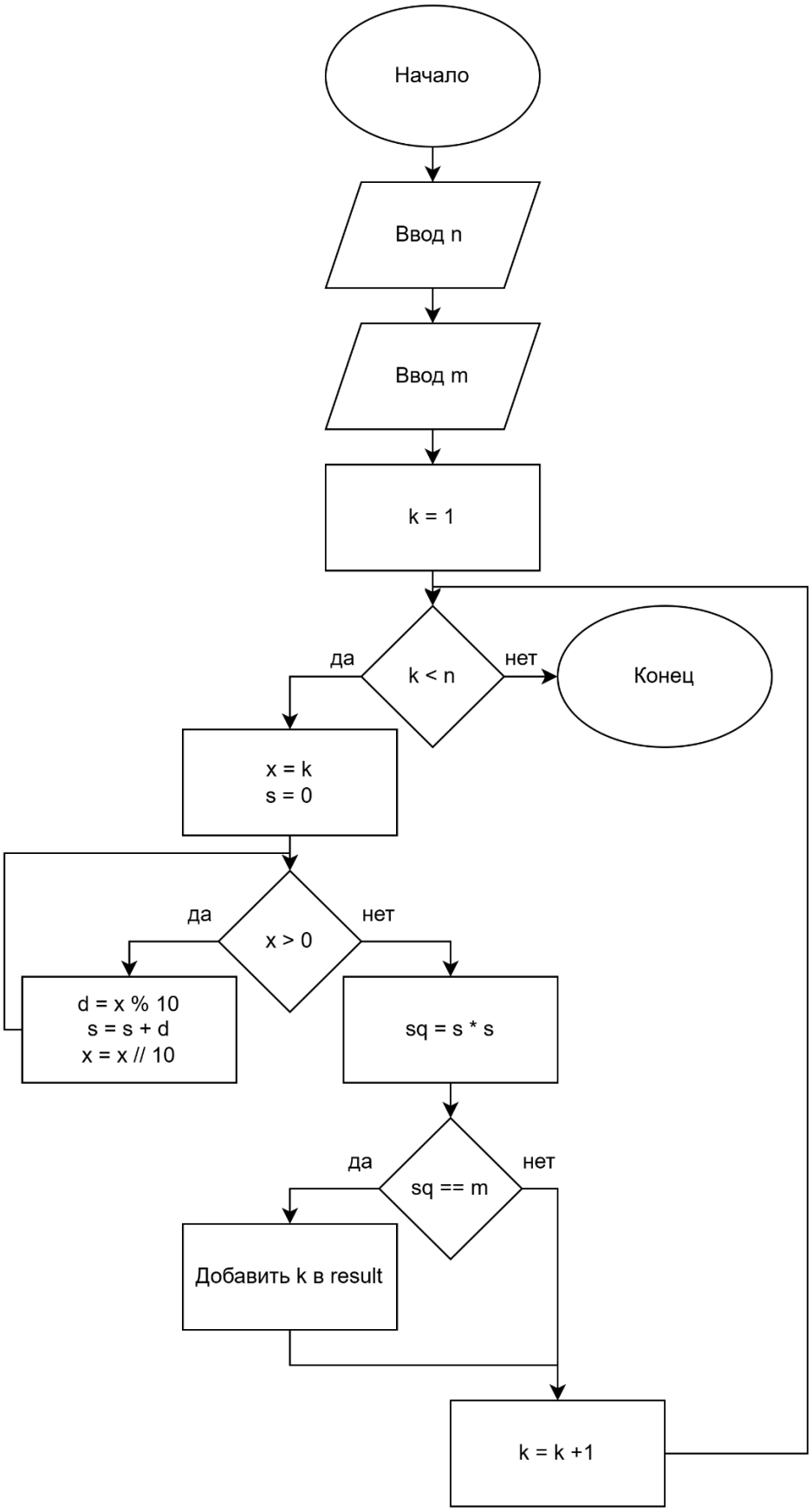


Рисунок 1.1 — Блок-схема алгоритма

1.3 Описание блок-схемы

На вход подаются n (граница) и m (искомый квадрат суммы цифр). Переменной k присваивается 1. Далее цикл: пока k < n — берём рабочую копию x = k и обнуляем накопитель суммы цифр s; пока x > 0, извлекаем последнюю цифру d = x mod 10, прибавляем её к s и отбрасываем цифру (x = x div 10). Когда все цифры разобраны, считаем sq = s*s и сравниваем с m: если равны — k добавляется в результат. Затем k увеличивается на 1, и цикл повторяется. Когда k достигает n, работа завершается.

1.4 Компонентный состав (аппаратные ресурсы)

Таблица 1.1 — Компонентный состав

Имя	Ресурс
PC	Счётчик команд
PROG	Память команд
RG_COM	Регистр команд
RF	Регистровый файл (16 регистров, [0:15])
MEM	Память данных
ALU	АЛУ
REG_A, REG_B	Регистры-операнды АЛУ

1.5 АСК — архитектура системы команд

Используются только команды, задействованные в программе (полный набор с занятия шире: LTM и MTR здесь не понадобились, так как входные данные — не массив, а два числа).

В системе команд намеренно нет операций "равно", деления/остатка и умножения — АЛУ умеет только сложение, вычитание и сравнение "меньше". Эти операции реализованы программно, циклами из базовых команд:

— равенство (sq = m): два JMPL подряд — если sq<m или m<sq, значит не равны; если ни то ни другое — равны;

— остаток и частное от деления на 10: цикл "пока r ≥ 10: r = r-10, q = q+1" — то, что осталось (r<10), это остаток, q — частное;

— возведение в квадрат (s*s): цикл, в котором s прибавляется к накопителю s раз (SUM в цикле).

Таблица 1.1 — Компонентный состав

Команда	Функционал	Арг. 1	Арг. 2	Действие
LTR	Запись числа в регистр	литерал	r1	RF[r1] = литерал
SUB	Вычитание	r1, r2	r3	RF[r3] = RF[r1] - RF[r2]
SUM	Сложение	r1, r2	r3	RF[r3] = RF[r1] + RF[r2]
INCR	Инкремент	r1	-	RF[r1] = RF[r1] + 1
JMPL	Переход, если меньше	r1, r2	addr	если RF[r1]<RF[r2]: PC=addr, иначе PC++
JMP	Безусловный переход	addr	-	PC = addr
RTM	Регистр -> память (косвенно)	r1, r2	-	MEM[RF[r1]] = RF[r2]

1.6 Микроархитектура (команды по тактам)

Таблица 1.2 — Микроархитектура

Команда	Fetch	Decode	Execute
LTR	RG_COM=PROG[PC]	REG_A=лит REG_B=r1	RF[REG_B]=REG_A PC=PC+1
SUB	RG_COM=PROG[PC]	REG_A=RF[r1] REG_B=RF[r2]	RF[r3]=REG_A-REG_B PC=PC+1
SUM	RG_COM=PROG[PC]	REG_A=RF[r1] REG_B=RF[r2]	RF[r3]=REG_A+REG_B PC=PC+1
INCR	RG_COM=PROG[PC]	REG_A=RF[r1] REG_B=1	RF[r1]=REG_A+REG_B PC=PC+1
JMPL	RG_COM=PROG[PC]	REG_A=RF[r1] REG_B=RF[r2]	если REG_A<REG_B: PC=addr, иначе PC=PC+1
JMP	RG_COM=PROG[PC]	не используется	PC=addr_to_jmp (берётся прямо из команды, минуя REG_A/REG_B)
RTM	RG_COM=PROG[PC]	REG_A=r2 REG_B=RF[r1]	MEM[REG_B]=RF[REG_A] PC=PC+1

1.7 Основные размерности

Literal = 16 bit REG_A = 16 bit REG_B = 16 bit

[15:0] RF [0:15] адрес регистра 4 бита

[15:0] MEM [0:1023] адрес в памяти 10 бит

Код операции 4 бита

[29:0] PROG [0:1023]

PC = 10 bit RG_COM = 30 bit

Длина команд: LTR - 24 бита, JMPL - 22 бита, SUB/SUM - 16 бит, RTM - 12 бит, JMP - 14 бит, INCR - 8 бит.

1.8 Ассемблер

r0 = n

r1 = m

r2 = k

r4 = x

r5 = s (сумма цифр)

r6 = 10 (константа)

r8 = r (остаток)

r9 = q (частное)

r10 = 0 (константа)

r11 = sq (квадрат суммы)

r12 = i2 (счётчик для квадрата)

r13 = addr (адрес записи результата)

r14 = cnt (кол-во найденных чисел)

r15 = outbase (адрес массива результатов = 10)

Листинг 1.2 — Код алгоритма на Ассемблере

```
 0: start:          LTR   r1=0 lit=0        // n (задаётся при запуске)
 1:                 LTR   r1=1 lit=0        // m (задаётся при запуске)
 2:                 LTR   r1=2 lit=1        // k = 1
 3:                 LTR   r1=6 lit=10       // const10 = 10
 4:                 LTR   r1=10 lit=0       // const0 = 0
 5:                 LTR   r1=14 lit=0       // cnt = 0
 6:                 LTR   r1=15 lit=10      // outbase = 10
 7: outer_test:     JMPL  r1=2 r2=0 -> 9    // k < n ?
 8:                 JMP    -> 36            // иначе выход
 9: outer_body:     SUB   r1=2 r2=10 r3=4   // x = k
10:                 SUB   r1=10 r2=10 r3=5  // s = 0
11: digit_loop:     JMPL  r1=10 r2=4 -> 13  // x > 0 ?
```

Продолжение Листинга 1.2

```
12:                 JMP    -> 22            // иначе сумма готова
13: extract_digit:  SUB   r1=10 r2=10 r3=9  // q = 0
14:                 SUB   r1=4 r2=10 r3=8   // r = x
15: div_test:       JMPL  r1=8 r2=6 -> 19   // r < 10 ?
16:                 SUB   r1=8 r2=6 r3=8    // r -= 10
17:                 INCR  r1=9              // q++
18:                 JMP    -> 15
19: digit_done:     SUM   r1=5 r2=8 r3=5    // s += r
20:                 SUB   r1=9 r2=10 r3=4   // x = q
21:                 JMP    -> 11
22: have_sum:       SUB   r1=10 r2=10 r3=11 // sq = 0
23:                 SUB   r1=10 r2=10 r3=12 // i2 = 0
24: sq_test:        JMPL  r1=12 r2=5 -> 26  // i2 < s ?
25:                 JMP    -> 29
26: sq_body:        SUM   r1=11 r2=5 r3=11  // sq += s
27:                 INCR  r1=12             // i2++
28:                 JMP    -> 24
29: check_eq:       JMPL  r1=11 r2=1 -> 34  // sq<m -> не равно
30:                 JMPL  r1=1 r2=11 -> 34  // m<sq -> не равно
31: store:          SUM   r1=15 r2=14 r3=13 // addr = outbase+cnt
32:                 RTM   r1=13 r2=2        // MEM[addr] = k
33:                 INCR  r1=14             // cnt++
34: next_k:         INCR  r1=2              // k++
35:                 JMP    -> 7
36: halt:           JMP    -> 36            // конец
```

1.9 Python-эмулятор

Листинг 1.3 — Код на Python-эмулятор

```
    from dataclasses import dataclass

COPS = {"LTR","SUB","INCR","JMP","JMPL","SUM","RTM"}

@dataclass
class Command:
    cop: str
    r1: int = 0
    r2: int = 0
    r3: int = 0
    lit: int = 0
    addr_to_jmp: int = 0
    def __post_init__(self):
        if self.cop not in COPS:
            raise ValueError(f"Unknown {self.cop}")

class Emulator:
    def __init__(self, prog):
        self.PROG = prog
        self.PC = 0
        self.reg_A = 0
        self.reg_B = 0
        self.RG_COM = 0
        self.RF = [0]*16
        self.MEM = [0]*64

    def fetch(self):
        self.RG_COM = self.PROG[self.PC]

    def decode(self):
        cmd = self.RG_COM
```

Продолжение Листинга 1.3

```
        if cmd.cop == "LTR":
            self.reg_A, self.reg_B = cmd.lit, cmd.r1
        elif cmd.cop == "INCR":
            self.reg_A, self.reg_B = self.RF[cmd.r1], 1
        elif cmd.cop == "RTM":
            self.reg_A, self.reg_B = cmd.r2, self.RF[cmd.r1]
        elif cmd.cop in ("SUB","SUM","JMPL"):
            self.reg_A, self.reg_B = self.RF[cmd.r1], self.RF[cmd.r2]

    def execute(self):
        cmd = self.RG_COM
        if cmd.cop == "LTR":
            self.RF[self.reg_B] = self.reg_A; self.PC += 1
        elif cmd.cop == "SUM":
            self.RF[cmd.r3] = self.reg_A + self.reg_B; self.PC += 1
        elif cmd.cop == "SUB":
            self.RF[cmd.r3] = self.reg_A - self.reg_B; self.PC += 1
        elif cmd.cop == "JMPL":
            self.PC = cmd.addr_to_jmp if self.reg_A < self.reg_B else
self.PC+1
        elif cmd.cop == "INCR":
            self.RF[cmd.r1] = self.reg_A + self.reg_B; self.PC += 1
        elif cmd.cop == "JMP":
            self.PC = cmd.addr_to_jmp
        elif cmd.cop == "RTM":
            self.MEM[self.reg_A] = self.reg_B; self.PC += 1

PROG = [
    Command("LTR", r1=0, lit=0),                    # 0: n
    Command("LTR", r1=1, lit=0),                    # 1: m
    Command("LTR", r1=2, lit=1),                    # 2: k = 1
    Command("LTR", r1=6, lit=10),                   # 3: const10
    Command("LTR", r1=10, lit=0),                   # 4: const0
    Command("LTR", r1=14, lit=0),                   # 5: cnt = 0
    Command("LTR", r1=15, lit=10),                  # 6: outbase
    Command("JMPL", r1=2, r2=0, addr_to_jmp=9),     # 7: k<n?
    Command("JMP", addr_to_jmp=36),                 # 8: halt
    Command("SUB", r1=2, r2=10, r3=4),              # 9: x=k
    Command("SUB", r1=10, r2=10, r3=5),             # 10: s=0
    Command("JMPL", r1=10, r2=4, addr_to_jmp=13),   # 11: x>0?
    Command("JMP", addr_to_jmp=22),                 # 12
    Command("SUB", r1=10, r2=10, r3=9),             # 13: q=0
    Command("SUB", r1=4, r2=10, r3=8),              # 14: r=x
    Command("JMPL", r1=8, r2=6, addr_to_jmp=19),    # 15: r<10?
    Command("SUB", r1=8, r2=6, r3=8),               # 16: r-=10
    Command("INCR", r1=9),                          # 17: q++
    Command("JMP", addr_to_jmp=15),                 # 18
    Command("SUM", r1=5, r2=8, r3=5),               # 19: s+=r
    Command("SUB", r1=9, r2=10, r3=4),              # 20: x=q
    Command("JMP", addr_to_jmp=11),                 # 21
    Command("SUB", r1=10, r2=10, r3=11),            # 22: sq=0
    Command("SUB", r1=10, r2=10, r3=12),            # 23: i2=0
    Command("JMPL", r1=12, r2=5, addr_to_jmp=26),   # 24: i2<s?
    Command("JMP", addr_to_jmp=29),                 # 25
    Command("SUM", r1=11, r2=5, r3=11),             # 26: sq+=s
    Command("INCR", r1=12),                         # 27: i2++
    Command("JMP", addr_to_jmp=24),                 # 28
    Command("JMPL", r1=11, r2=1, addr_to_jmp=34),   # 29: sq<m?
    Command("JMPL", r1=1, r2=11, addr_to_jmp=34),   # 30: m<sq?
    Command("SUM", r1=15, r2=14, r3=13),            # 31: addr
    Command("RTM", r1=13, r2=2),                    # 32: MEM[addr]=k
    Command("INCR", r1=14),                         # 33: cnt++
    Command("INCR", r1=2),                          # 34: k++
```

Продолжение Листинга 1.3

```
    Command("JMP", addr_to_jmp=7),                  # 35
    Command("JMP", addr_to_jmp=36),                 # 36: halt
]

def run(n, m):
    prog = list(PROG)
    prog[0] = Command("LTR", r1=0, lit=n)
    prog[1] = Command("LTR", r1=1, lit=m)
    e = Emulator(prog)
    while e.PC != 36:
        e.fetch(); e.decode(); e.execute()
    cnt = e.RF[14]
    return [e.MEM[10+i] for i in range(cnt)]

if __name__ == '__main__':
    print(run(30, 16))   # -> [4, 13, 22]
```

2 ТЕСТИРОВАНИЕ

2.1 Результаты тестирования

Программа проверена на нескольких наборах входных данных, результат эмулятора сверен с независимым расчётом (перебор в Python).

Таблица 1.3 — Таблтца с результатами тестирования

n	m	Результат эмулятора	Ожидалось	Итог
20	1	[1, 10]	[1, 10]	совпало
30	16	[4, 13, 22]	[4, 13, 22]	совпало
5	100	[]	[]	совпало
100	25	[5, 14, 23, 32, 41, 50]	[5, 14, 23, 32, 41, 50]	совпало
1	1	[]	[] (граничный случай)	совпало
2	1	[1]	[1]	совпало

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы был разработан алгоритм решения индивидуального варианта задачи, построена блок-схема, определён компонентный состав процессорного ядра и архитектура системы команд, включающая семь команд: LTR, SUB, SUM, INCR, JMPL, JMP и RTM. Для каждой команды описаны такты выполнения (Fetch, Decode, Execute) и разрядность полей.

На основе разработанной аналитики реализован программный эмулятор процессорного ядра на языке Python, а также соответствующая ему ассемблерная программа из 37 команд. Деление числа на цифры и возведение суммы цифр в квадрат реализованы через циклы с использованием только вычитания, сложения и инкремента, что соответствует ограничениям заданной системы команд. Проверка равенства двух значений (квадрата суммы цифр и числа m), при отсутствии в системе команд соответствующей операции, реализована через два последовательных сравнения "меньше".

Работа эмулятора протестирована на шести наборах входных данных, включая граничные случаи (n = 1, а также значения m, при которых результат должен быть пустым). Во всех случаях результат работы эмулятора совпал с результатом независимого расчёта, что подтверждает корректность разработанной архитектуры и её программной реализации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Смирнов С.С. Информатика [Электронный ресурс]: Методические указания по выполнению практических и лабораторных работ / С.С. Смирнов — М., МИРЭА — Российский технологический университет, 2018. — 1 электрон. опт. диск (CD-ROM).

2. Тарасов И.Е. ПЛИС Xilinx. Языки описания аппаратуры VHDL и Verilog, САПР, приемы проектирования. — М.: Горячая линия — Телеком, 2021. — 538 с.: ил.

3. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие / Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).

4. Антик М.И. Математическая логика и программирование в логике [Электронный ресурс]: Учебное пособие / Антик М.И., Бражникова Е.В.— М.: МИРЭА – Российский технологический университет, 2018. — 1 электрон. опт. диск (CD-ROM).

5. Жемчужникова Т.Н. Конспект лекций по дисциплине «Архитектура вычислительных машин и систем» — URL: https://drive.google.com/file/d/12OAi2_axJ6mRr4hCbXs-mYs8Kfp4YEfj/view?usp=sharing (Дата обращения: 23.02.2026).

6. Антик М.И. Теория автоматов в проектировании цифровых схем [Электронный ресурс]: Учебное пособие / Антик М.И., Казанцева Л.В. — М.: МИРЭА – Российский технологический университет, 2020. — 1 электрон. опт. диск (CD-ROM).

7. Орлов С.А. Организация ЭВМ и систем: Учебник для вузов. 4-е изд. — СПб.: Питер, 2018. — 688 с.: ил.

8. Шустов М.А. Цифровая схемотехника. Основы построения. — СПб.: Наука и Техника, 2018. — 320 с.: ил.

9. Рафиков Р. А. Электронные сигналы и цепи. Цифровые сигналы и устройства: Учебное пособие. — СПб.: Издательство «Лань», 2016. — 320 с., ил. — (Учебники для вузов. Специальная литература).

10. Угрюмов Е. П. Цифровая схемотехника: учеб. пособие для вузов. — 3-е изд., перераб. и доп. — СПб.: БХВ-Петербург, 2010. — 816 с.: ил.